

LAB WALKTHROUGH

Publishing a Web App through a Shared Application Gateway

Azure | Hub-and-Spoke | App Gateway v2

Overview

In this lab you will expose your existing Azure Web App through the shared Application Gateway that lives in the hub virtual network. By the end of the lab, traffic that hits the public IP of the hub Application Gateway on the hostname `yourname.bloodhound.ro` will be routed to your Web App.

You will:

- Add a new backend pool that points to your Web App.
- Create an HTTP listener bound to your personal hostname on port 80.
- Add HTTP backend settings and wire everything together with a routing rule.
- Re-enable HTTP traffic on the Web App so it accepts the gateway's requests.

Important: Replace `yourname` everywhere in this document with the prefix assigned to you (for example, `john-webapp` and `john.bloodhound.ro`). The hub Application Gateway is shared between students, so be careful to only touch resources that belong to your hostname.

Prerequisites

- An Azure subscription with **Contributor** role on your spoke resource group.
- Your Web App deployed and running. Name format: `yourname-webapp`
- Read access on the hub resource group that contains `hub-app-gtw`.
- The DNS record `yourname.bloodhound.ro` already pointing to the hub Application Gateway's public IP (the trainer will confirm this).

Architecture at a glance

The flow you are building looks like this:

```

Internet → yourname.bloodhound.ro (DNS) → Public IP of hub-app-gtw
→ Listener (port 80, host = yourname.bloodhound.ro) → Routing rule
→ Backend pool (yourname-webapp.azurewebsites.net) → Backend HTTP
settings (port 80) → Azure Web App
  
```

Step 1 — Locate the hub Application Gateway

1. Sign in to the **Azure Portal** at <https://portal.azure.com>.
2. In the top search bar, type **hub-app-gtw** and open the Application Gateway resource.
3. Confirm with your trainer that you are looking at the correct gateway (it should be in the hub resource group, SKU `WAF_v2` or `Standard_v2`).

Tip: Pin the Application Gateway to your favorites — you will come back to it several times throughout this lab.

Step 2 — Create the backend pool

The backend pool tells the gateway where to send traffic. You will point it to the default hostname of your Web App.

1. In the hub-app-gtw blade, select **Backend pools** under *Settings*.
2. Click **+ Add**.
3. Fill in the backend pool with these values:

Setting	Value
Name	yourname-webapp-pool
Add backend pool without targets	No
Backend targets — Target type	IP address or FQDN
Target	yourname-webapp.azurewebsites.net

4. Click **Add** to save the pool.

Why FQDN and not IP? Azure Web Apps sit behind a multi-tenant front-end with dynamic IPs. Always use the FQDN so the gateway resolves the current IP at request time.

Home > Load balancing and content delivery | Application gateways > hub-app-gtw | Backend pools

Edit backend pool ...

A backend pool is a collection of resources to which your application gateway can send traffic. A backend pool can contain virtual machines, virtual machines scale sets, IP addresses, domain names, or an App Service.

Name

dorin-web-backend-pool

Add backend pool without targets

Yes

No

Backend targets

1 item

Target type	Target
App Services	dorinweb-adb3czfbcvhuhnbm.westeurop...  ...
IP address or FQDN	

Step 3 — Create the HTTP listener

The listener decides which incoming requests this routing rule applies to. You will create a multi-site listener that only matches your hostname, so multiple students can share the same gateway and port without colliding.

1. Still in `hub-app-gtw`, open **Listeners** → **+ Add listener**.
2. Configure the listener:

Setting	Value
Listener name	yourname-listener
Frontend IP	Public (the existing public frontend of the gateway)
Protocol	HTTP
Port	80
Listener type	Multi site
Host type	Single
Host name	yourname.bloodhound.ro

3. Leave *Custom error pages* empty and click **Add**.

Why multi-site? A Basic (single-site) listener owns the entire port-80 frontend. Multi-site listeners use Host headers to share the same port between many backends — which is exactly what a shared classroom gateway needs.

dorin-web-listener ...

hub-app-gtw

Listener name ⓘ

dorin-web-listener

Frontend IP * ⓘ

Public

Protocol ⓘ

☒ HTTP
 ☐ HTTPS
 ☐ TCP
 ☐ TLS

Port * ⓘ

80

Associated rule

[web-dorin-routing](#)

Listener type ⓘ

☐ Basic
 ☒ Multi site

Host type ⓘ

☒ Single
 ☐ Multiple/Wildcard

Host name * ⓘ

webdorin.bloodhound.ro

Custom error pages

Show customized error pages for different response codes generated by Application Gateway. This section lets you configure Listener-specific error pages. [Learn more](#)

Please verify that the url(s) being added here is reachable from your application gateway using the [connection troubleshoot](#) tool to prevent any deployment error.

Bad Gateway - 502

Enter Html file URL

Forbidden - 403

Enter Html file URL

Step 4 — Create the HTTP backend settings

Backend settings describe how the gateway talks to the backend: protocol, port, timeouts, and — crucially for App Service — which hostname to send.

1. Open **Backend settings** → **+ Add**.
2. Configure the backend settings:

Setting	Value
Backend settings name	yourname-http-settings
Backend protocol	HTTP

Setting	Value
Backend port	80
Cookie-based affinity	Disable
Connection draining	Disable
Request time-out (seconds)	30
Override with new host name	Yes
Host name override	Pick host name from backend target
Use custom probe	No (use default)

3. Click **Add**.

Critical setting: If you do not override the host header, the gateway forwards yourname.bloodhound.ro to App Service. App Service does not know that hostname and returns HTTP 404: Web app not found. Picking the hostname from the backend target sends yourname-webapp.azurewebsites.net instead, which App Service does recognize.

Add Backend setting

Backend port *

80 ✓

Additional backend settings

Cookie-based affinity ⓘ
☐ Enable ☒ Disable

Connection draining ⓘ
☐ Enable ☒ Disable

Dedicated backend connection ⓘ
☐ Enable ☒ Disable

Request time-out (seconds) * ⓘ
 20

Override backend path ⓘ

Override hostname

By default, the Application Gateway sends the same HTTP host header to the backend as it receives from the client. If your backend application/service requires a specific host value, you can override it using this setting.

Override with new host name
☒ Yes ☐ No

Host name override

☒ Pick host name from backend target
☐ Override with specific domain name

i If the backend service is a multi-tenant Azure service such as App Services, Functions, or Portal Apps, we recommend using [Custom domain method](#), instead of overriding the hostname. Using override host name with default domains (azurewebsites.net, azuremicroservices.io, etc.) is good only for the basic tests and operations.

Step 5 — Create the routing rule

The rule glues listener, backend pool, and backend settings together.

1. Open **Rules** → **+ Request routing rule**.
2. On the Listener tab:

Setting	Value
Rule name	yourname-rule
Priority	Any free number (e.g. 100). Lower number = higher priority.
Listener	yourname-listener

3. Switch to the **Backend targets** tab.
4. Configure:

Setting	Value
Target type	Backend pool
Backend target	yourname-webapp-pool
Backend settings	yourname-http-settings

5. Click **Add** and wait for the gateway to update. This usually takes 1–3 minutes.

Pick a unique priority All rules on a shared v2 gateway must have unique priorities. If you get an error, just bump your number by one.

web-dorin-routing hub-app-gtw ×

Configure a routing rule to send traffic from a given frontend IP address to one or more backend targets. A routing rule must contain a listener and at least one backend target.

Rule name

Priority * ⓘ

* Listener * Backend targets

Choose a backend pool to which this routing rule will send traffic. You will also need to specify a set of Backend settings that define the behavior of the routing rule. ⓘ

Target type ☒ Backend pool ☐ Redirection

Backend target * ⓘ

Backend settings * ⓘ

Step 6 — Enable HTTP on the Web App

By default Azure Web Apps redirect all HTTP traffic to HTTPS. Since your listener and backend settings are both HTTP on port 80, you need to allow plain HTTP for this lab.

1. Navigate to your Web App `yourname-webapp`.
2. Open *Settings* → **Configuration** → **General settings** tab.
3. Change the following:

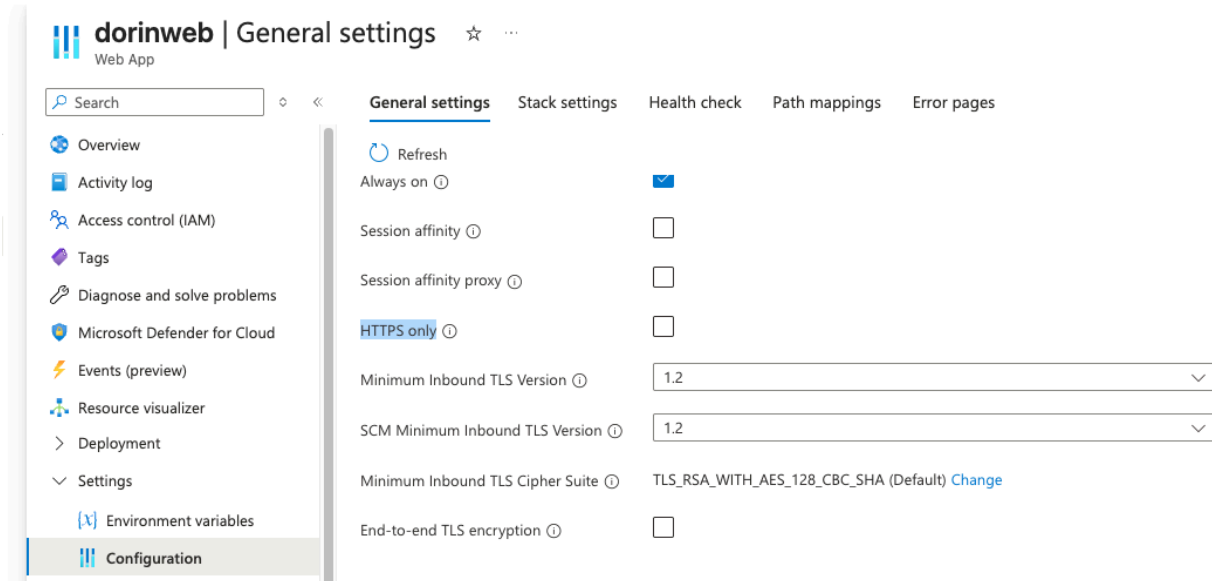
Setting	Value
HTTPS Only	Off
Minimum Inbound TLS Version	Leave as 1.2 (not used for HTTP, no impact)

4. Click **Save** at the top of the blade and confirm the restart prompt.

Production note: Disabling HTTPS Only is acceptable for this lab because the Application Gateway sits in front of the Web App. In a production design you would terminate TLS at the gateway (HTTPS listener) and either keep HTTP between gateway and Web App over the private link, or use HTTPS end-to-end.

Optional — restrict the Web App so only the gateway can reach it:

- Open **Networking** → **Access restrictions** on the Web App.
- Add a rule that allows only the hub Application Gateway's public IP (or its subnet).



Step 7 — Validate

7.1 Backend health

5. In hub-app-gtw, open **Backend health** under *Monitoring*.
6. Find the row matching yourname-webapp-pool / yourname-http-settings.
7. Status should be **Healthy** (green).

7.2 End-to-end test

1. From your local machine, open a browser and visit:
2. `http://yourname.bloodhound.ro`
3. You should see the home page of your Web App.
4. From a terminal you can also confirm with curl:
5. `curl -v http://yourname.bloodhound.ro`

Look for HTTP/1.1 200 in the response headers.

Troubleshooting

502 Bad Gateway

- Backend health is Unhealthy — usually means the Web App is stopped, or HTTPS Only is still on (the probe gets a 301 and the gateway marks the backend unhealthy).
- Fix: re-check Step 6 and confirm `HTTPS Only = Off`.

404: Web app not found

- Host header override is missing or wrong. Re-open the backend settings and confirm `Override with new host name = Yes` and `Pick host name from backend target` is selected.

ERR_NAME_NOT_RESOLVED in browser

- The DNS record for your hostname is missing. Confirm with the trainer that `yourname.bloodhound.ro` resolves to the gateway's public IP.
- Verify from your machine: `nslookup yourname.bloodhound.ro`.

Listener overlap error when creating the listener

- Another student is using the same hostname. Check the spelling — it must match the DNS record exactly.

Step 8 — Create and attach your own WAF policy

The hub Application Gateway is a WAF SKU, which means each listener can have its own WAF policy. You will create a per-student policy, attach it to your listener only, put it in Prevention mode, and confirm it blocks a script-injection attempt.

Why per-listener? A WAF policy attached at the listener level affects only that listener's traffic. Other students sharing this gateway will not be impacted by your rules, false positives, or test payloads.

8.1 Create the WAF policy

1. In the Azure Portal search bar, type **WAF policies** and open *Web Application Firewall policies (WAF)*.
2. Click **+ Create**.
3. On the Basics tab, fill in:

Setting	Value
Policy for	Regional WAF (Application Gateway)
Subscription	Same as the hub Application Gateway
Resource group	Your own spoke resource group (not the hub RG)
Policy name	<code>yourname-waf-policy</code>
Location	Same region as hub-app-gtw
Policy state	Enabled

Setting	Value
Policy mode	Detection (we will switch to Prevention later)

Why start in Detection? Detection mode only logs requests that match a rule — it does not block. This lets you sanity-check the policy is attached correctly before you start blocking real traffic. Always promote to Prevention only after you have a baseline.

- Click **Next: Managed rules**.
- Accept the default rule set — at the time of writing, the recommended default is the Microsoft Default Rule Set (DRS) 2.1 or newer. Do not disable any managed rules.
- Click **Next: Policy settings** and confirm:

Setting	Value
Inspect request body	Enabled
Max request body size (KB)	128 (default)
File upload limit (MB)	100 (default)

- Skip Custom rules and Association tabs for now — we'll attach the policy explicitly in the next section so we can target the listener (not the whole gateway).
- Click **Review + create** → **Create**. Wait for the deployment to finish.

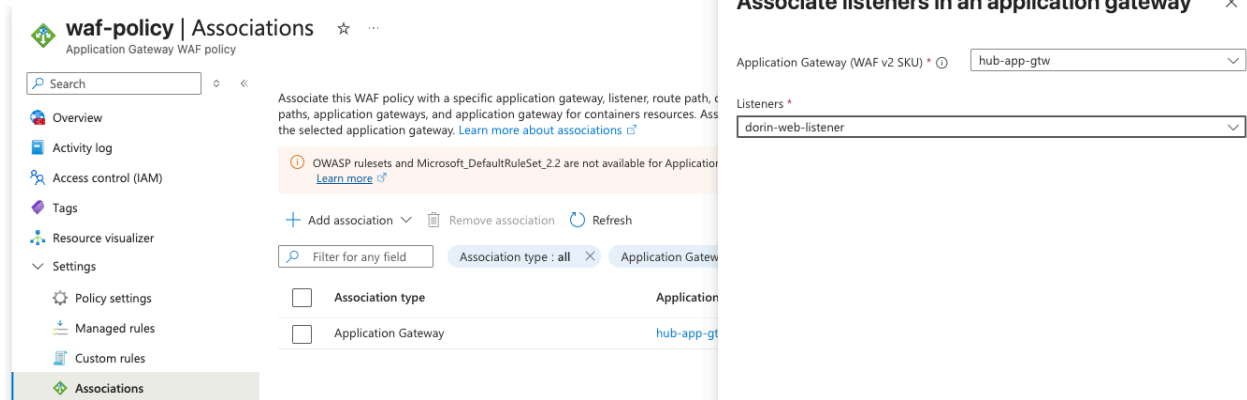
7.2 Associate the policy with your listener

This is the part most students miss: you can either attach the WAF policy from the policy blade or from the listener. We will do it from the policy blade so the scope choice is explicit.

- Open your new yourname-waf-policy.
- Go to **Associated application gateways** under *Settings*.
- Click **Associate** (or **+ Associate a gateway** depending on portal version).
- Set the scope:

Setting	Value
Association type	HTTP Listener
Application gateway	hub-app-gtw
Listener	yourname-listener

- Click **Add** or **Save**. The gateway will update — give it 2–4 minutes.



Critical: Do NOT associate at the Application gateway scope. That would replace the gateway-wide policy (if one exists) and affect every other student on the gateway. The Listener scope is the only safe option for a shared lab environment.

7.3 Confirm the policy is attached

1. Open `hub-app-gtw` → **Listeners** → click `yourname-listener`.
2. In the listener's properties, you should see a **Firewall Policy** field showing `yourname-waf-policy`.
3. Browse to `http://yourname.bloodhound.ro` one more time — the site should still load normally. Detection mode does not block anything.

7.4 Switch to Prevention mode

1. Go back to `yourname-waf-policy`.
2. Open **Policy settings** under *Settings*.
3. Change:

Setting	Value
Mode	Prevention

4. Click **Save**. The gateway will reload the policy — typically under 60 seconds.

7.5 Test with a malicious payload

Now you will send a classic reflected-XSS test string through your hostname. In Prevention mode, the WAF should intercept it and return an HTTP 403 page instead of forwarding the request to your Web App.

Lab use only: This is a deliberately crafted test payload used to verify the WAF, not an attack against any real system. Do not run this kind of test against systems you do not own.

9. In your browser, paste the following URL into a new tab:

```
http://yourname.bloodhound.ro/?q=<script>alert()</script>
```

10. Expected result: an **HTTP 403 Forbidden** page from the Application Gateway — not your Web App's home page.
11. Repeat the test from a terminal so you can see the exact status code and headers:

```
curl -v "http://yourname.bloodhound.ro/?q=<script>alert()</script>"
```

You should see something like:

```
< HTTP/1.1 403 Forbidden< Server: Microsoft-Azure-Application-Gateway/v2
```

Why the browser URL-encodes the payload Modern browsers will encode < and > to %3C and %3E in the address bar. The WAF inspects the decoded value, so the policy still triggers. If you want to see the raw payload sent on the wire, use the curl example above.

Cleanup

If your trainer asks you to undo your changes, remove resources from the gateway in this order to avoid dependency errors:

1. Rule — yourname-rule
2. Listener — yourname-listener
3. Backend settings — yourname-http-settings
4. Backend pool — yourname-webapp-pool

Leave the Web App as-is unless told otherwise.

What you learned

- How to add a tenant (backend pool + listener + rule) to a shared Application Gateway without disrupting other students.
- Why multi-site listeners are the standard pattern for shared gateways.
- Why App Service requires a host header override when fronted by Application Gateway.
- How HTTPS Only on App Service interacts with HTTP probes from the gateway.